

Herencia

10.1 Introducción

La herencia es un concepto fundamental en la programación orientada a objetos que establece una relación esencial entre clases. Este mecanismo se aplica cuando existen clases que, aunque comparten similitudes notables, no son lo suficientemente idénticas como para ser consideradas como una única clase. En esta situación, la herencia entra en juego para vincular estas clases de manera que los objetos pertenecientes a ambas puedan ser utilizados de manera equivalente, como si efectivamente fueran instancias de la misma clase.

El proceso de herencia posibilita la creación de una nueva clase basada en una clase preexistente, sin la necesidad de duplicar y repetir el código de la clase original. De esta manera, se evita la tarea de copiar y pegar la implementación de una clase para crear una nueva, lo que redundaría en un mantenimiento engorroso y potencialmente propenso a errores.

La herencia implica que una clase derivada (o subclase) hereda propiedades y métodos de una clase base (o superclase). La subclase puede agregar nuevas funcionalidades o modificar las existentes, pero también hereda la estructura y el comportamiento de la superclase.

Esto es útil porque permite organizar y abstraer el código de manera eficiente. Por ejemplo, si se está desarrollando un programa para modelar vehículos, se podría tener una clase base llamada “Vehículo” con propiedades y métodos genéricos como “acelerar” y “frenar”. Luego, se pueden crear

subclases tales como “Automóvil” y “Motocicleta” que heredan estas características básicas y agregan sus propias particularidades, ya sea agregando nuevos atributos y métodos o modificando métodos heredados brindando una implementación específica para sus responsabilidades y necesidades.

En síntesis, la herencia permite tratar uniformemente a objetos de distintas clases relacionadas, evitando la duplicación de código y favoreciendo la reutilización de estructura y comportamiento común.

10.2 “Es un”

El concepto de la herencia en la programación orientada a objetos se manifiesta mediante la capacidad de definir una clase a partir de otra, estableciendo una relación específica entre ellas. En este contexto, una de las clases involucradas se denomina **clase base**, mientras que la otra se conoce como **clase derivada**.

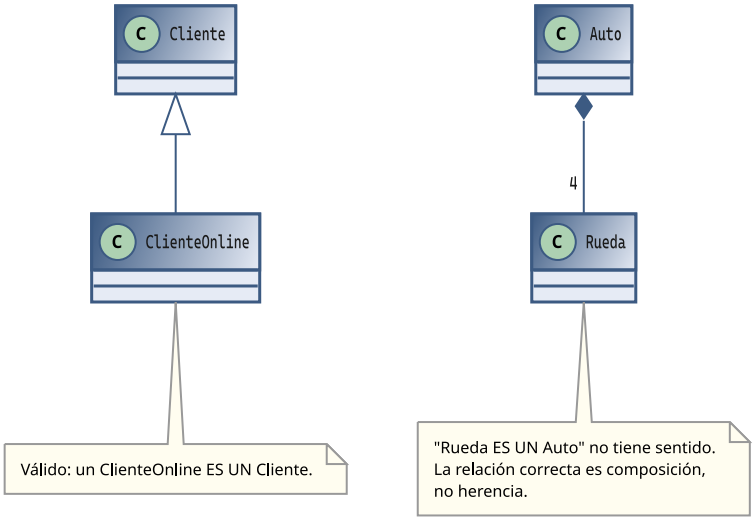
La clase derivada puede conceptualizarse como un caso especial o una variante de la clase base, lo que implica que todos los objetos pertenecientes a la clase derivada pueden ser considerados como objetos de la clase base. Esta relación se expresa de manera precisa con la afirmación: “un objeto de la clase derivada ES UN objeto de la clase base”. En general, es una práctica útil verificar las relaciones de herencia con esa frase: si la frase no tiene sentido, generalmente es síntoma de que las clases no deben estar relacionadas o lo hacen pero por otro tipo de relación.

Por ejemplo:

Clase base Cliente y Clase derivada ClienteOnline En este contexto, afirmar que “un cliente online ES UN cliente” es totalmente válido. La clase derivada ClienteOnline representa una variante específica de la clase base Cliente, pero todos los clientes en línea son, en última instancia, clientes en sí mismos.

Clase base Auto y Clase derivada Rueda La afirmación “una rueda ES UN auto” es inevitablemente incorrecta, como también lo es “un auto ES UNA rueda”. Esta situación pone en evidencia que tales clases no pueden estar relacionadas mediante herencia. Sin embargo, pueden estar relacionadas mediante algún otro tipo de relación, por caso, composición.

El siguiente diagrama contrasta ambos casos: a la izquierda, una relación de herencia válida (Cliente / ClienteOnline); a la derecha, una relación que no debe modelarse con herencia (Auto / Rueda no superan la prueba “ES UN”), sino con composición.



10.3 Implementación

10.3.1 Bloque class

Para implementar herencia en Python no se agregan muchos elementos sintácticos, en general las clases se redactan de una forma bastante similar a aquellas no involucradas en este tipo de relación.

En la clase base no se debe realizar ninguna modificación si se identifica la necesidad de una clase derivada. En la clase derivada se indica en la cabecera del bloque class, a continuación del nombre de la misma, el nombre de la clase base encerrado entre paréntesis. Retomando el ejemplo de Cliente y ClienteOnline de la sección anterior:

```
class ClienteOnline(Cliente):
```

10.3.2 Función super

A diferencia de `self`, que es un parámetro que cada método recibe automáticamente al ser invocado sobre una instancia, super **no es una referencia** disponible de antemano en el entorno del método, sino una función incorporada del lenguaje. Esto no tiene nada que ver con una herencia implícita desde alguna clase raíz, Python no expone `super` como un miembro que las instancias reciban por el solo hecho de pertenecer a una jerarquía de herencia, sino como una función del lenguaje que hay que llamar explícitamente, con paréntesis, cada vez que se la necesita.

Al invocarla sin argumentos dentro de un método de una clase derivada , `super()` construye y devuelve un objeto **proxy**: un objeto que no almacena datos propios, sino que delega la búsqueda de los miembros solicitados hacia la clase base, permitiendo acceder a los miembros heredados sin necesidad de nombrar explícitamente a dicha clase base.

Esta diferencia con `self` también explica una asimetría sintáctica que suele generar dudas: `self` nunca se "llama" –ya está ahí, recibido como parámetro–, mientras que `super()` sí se llama, precisamente porque es una función y no una referencia.

La función `super()` es ocasionalmente utilizada de forma errónea para acceder a cualquier miembro declarado en la clase base. Esto evidencia una notoria ignorancia del concepto de la herencia: todo miembro de la clase base es heredado y por lo tanto accesible mediante la referencia `self`.

El uso de `super()` debe focalizarse en un único escenario: cuando se sobreescribió un método en la clase derivada y se necesita invocar la versión del mismo método definida en la clase base. Este es el caso del método `__init__`, que al ser redefinido en la derivada no se ejecuta automáticamente, por lo que es necesario invocarlo explícitamente con `super()` para completar la inicialización. Pero también aplica a cualquier otro método que haya sido redefinido, como `__str__`, donde `super().__str__()` permite reutilizar la representación textual de la clase base y extenderla con nuevos atributos. Si en cualquiera de estos casos se intentara invocar el método con `self` desde dentro de la propia redefinición, se iniciaría una llamada recursiva; `super()` permite, en cambio, acceder a la versión original sin recursión.

La **redefinición** de un método (también llamada *override*) consiste en que una clase derivada reimplemente un método ya definido en su clase base para darle un comportamiento propio o especializado. Es, junto con el constructor,

el otro escenario habitual en el que super resulta indispensable: permite que la redefinición, además de su propio agregado, siga apoyándose en la implementación original de la base sin caer en una llamada recursiva sobre sí misma. La redefinición es una técnica central en el trabajo conjunto entre herencia y polimorfismo, y es en el capítulo de Polimorfismo donde se la trata y aplica en detalle.

10.3.3 Constructor

El método constructor de cada clase generalmente recibe parámetros para asignar valores iniciales a los atributos. Cuando se programa un constructor en una clase derivada, el caso más usual es el de recibir parámetros para inicializar los atributos de la propia clase y también los heredados.

A continuación el constructor de la derivada puede realizar una invocación explícita al constructor de la clase base a través de super. Es importante notar que los parámetros que recibe el constructor de la derivada normalmente **no** coinciden exactamente con los que se le reenvían a la base: la derivada suele recibir, además de los parámetros que la base necesita, otros propios para inicializar sus atributos adicionales, y es responsabilidad de la derivada seleccionar cuáles de ellos le corresponden a super().__init__().

Retomando y completando el ejemplo de Cliente y ClienteOnline, y agregando además una segunda clase derivada, ClientePresencial:

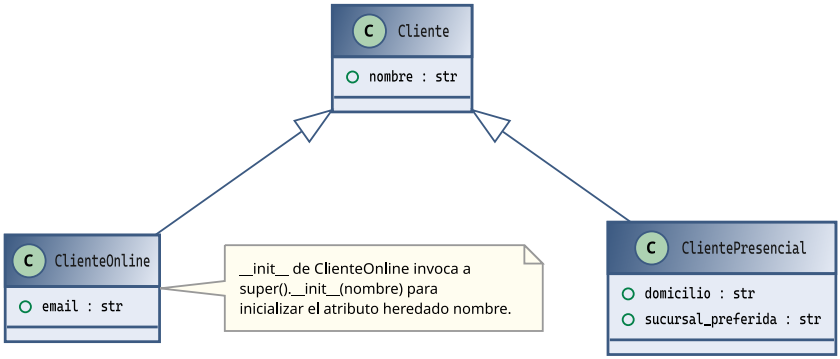
```
class Cliente:
    def __init__(self, nombre):
        self.nombre = nombre

class ClienteOnline(Cliente):
    def __init__(self, nombre, email):
        # nombre se reenvía tal cual a la base; email es un
        # parámetro propio de ClienteOnline, que Cliente desconoce.
        super().__init__(nombre)
        self.email = email

class ClientePresencial(Cliente):
    def __init__(self, nombre, domicilio, sucursal_preferida):
        # domicilio y sucursal_preferida tampoco existen en Cliente:
        # son propios de un cliente que opera en una sucursal física,
        # probablemente la más cercana a su domicilio.
        super().__init__(nombre)
        self.domicilio = domicilio
        self.sucursal_preferida = sucursal_preferida

# Crear un cliente online
cliente1 = ClienteOnline("Ana Gómez", "ana.gomez@correo.com")
```

```
# Crear un cliente presencial
cliente2 = ClientePresencial("Beto Ruiz", "Av. Gutierrez 742", "Sucursal Centro")
```





11.1 Introducción

11.1.1 Concepto

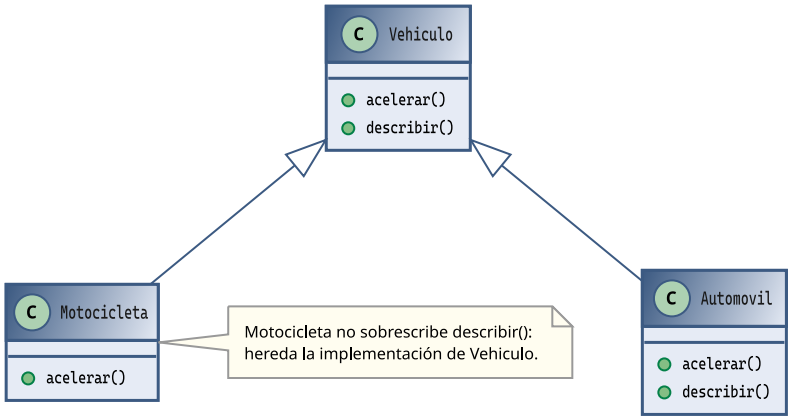
El polimorfismo es un principio fundamental de la programación orientada a objetos que permite que objetos de diferentes clases sean tratados como objetos de una clase común. Esta característica facilita que un mismo método pueda comportarse de manera diferente según el tipo específico del objeto que lo recibe, proporcionando flexibilidad y extensibilidad al código.

El término “polimorfismo” deriva del griego y significa “muchas formas”, lo cual describe precisamente su naturaleza: un mismo nombre de método puede tener múltiples implementaciones según la clase que lo contenga. En el contexto de la herencia, el polimorfismo se manifiesta cuando las clases derivadas redefinen métodos heredados de la clase base, proporcionando implementaciones específicas que se ajusten a sus particulares necesidades y características.

11.1.2 Sobreescritura de métodos

La sobreescritura de métodos es el mecanismo mediante el cual una clase derivada proporciona una implementación específica de un método que ya existe en su clase base. En Python, la sintaxis para sobreescribir un método es extremadamente sencilla: simplemente se define un método en la clase derivada con el mismo nombre que el método de la clase base que se desea redefinir.

No se requieren palabras clave especiales ni anotaciones adicionales.




```
class Vehiculo:
    def acelerar(self):
        print("El vehículo está acelerando")

    def describir(self):
        print("Este es un vehículo genérico")

class Automovil(Vehiculo):
    def acelerar(self):
        print("El automóvil acelera suavemente")

    def describir(self):
        print("Este es un automóvil con cuatro ruedas")

class Motocicleta(Vehiculo):
    def acelerar(self):
        print("La motocicleta acelera rápidamente")
```

El verdadero efecto del polimorfismo se aprecia cuando un mismo fragmento de código trata a objetos de distintas clases derivadas de manera uniforme, sin necesidad de saber con cuál de ellas está tratando en cada caso. Alcanza con saber que todos son `Vehiculo` para poder invocar `acelerar()` sobre cualquiera; la implementación que efectivamente se ejecuta depende del tipo concreto de cada objeto:

```
vehiculos = [Automovil(), Motocicleta()]

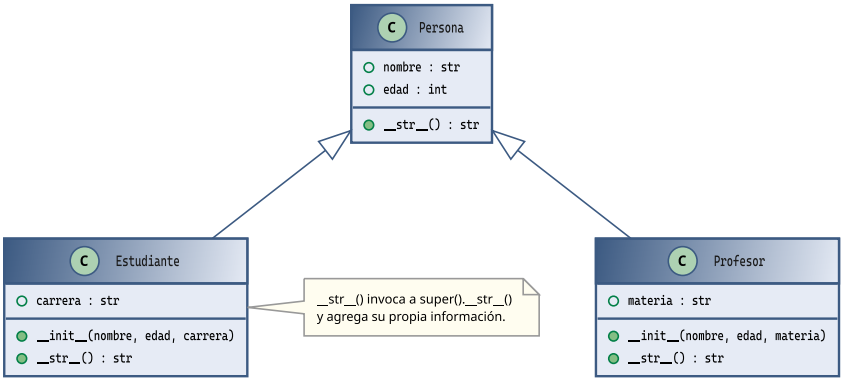
for vehiculo in vehiculos:
    vehiculo.acelerar()
```

Aunque el código del `for` es exactamente el mismo para los dos elementos de la lista —`vehiculo.acelerar()`—, el mensaje se envía en cada vuelta del ciclo hacia la implementación correspondiente a la clase real del objeto: la de `Automovil` en la primera iteración, la de `Motocicleta` en la segunda. Esto es lo que distingue al polimorfismo de simplemente "tener cada clase su propio método": lo relevante no es que existan varias implementaciones, sino que el código que las invoca puede ser único e ignorar por completo cuál de ellas se va a ejecutar.

11.1.3 Uso de super

El método especial `__str__` es particularmente útil para demostrar el uso de `super()` en el contexto del polimorfismo, ya que frecuentemente es necesario extender la representación de cadena de la clase base en lugar de reemplazarla completamente.

Cuando se sobreescribe el método `__str__` en una clase derivada, es común utilizar `super()` para invocar la implementación de la clase base y luego agregar información específica de la clase derivada. Esto permite construir representaciones de cadena más completas y jerárquicas.




```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __str__(self):
        return f"Persona: {self.nombre}, {self.edad} años"

class Estudiante(Persona):
    def __init__(self, nombre, edad, carrera):
```

```
        super().__init__(nombre, edad)
        self.carrera = carrera

    def __str__(self):
        return f"{super().__str__()}, Carrera: {self.carrera}"

class Profesor(Persona):
    def __init__(self, nombre, edad, materia):
        super().__init__(nombre, edad)
        self.materia = materia

    def __str__(self):
        return f"{super().__str__()}, Materia: {self.materia}"
```

En este ejemplo, las clases `Estudiante` y `Profesor` extienden la representación de cadena de la clase `Persona` agregando información específica. El uso de `super().__str__()` permite reutilizar la lógica de la clase base y evitar la duplicación de código.

11.2 Clases y métodos abstractos

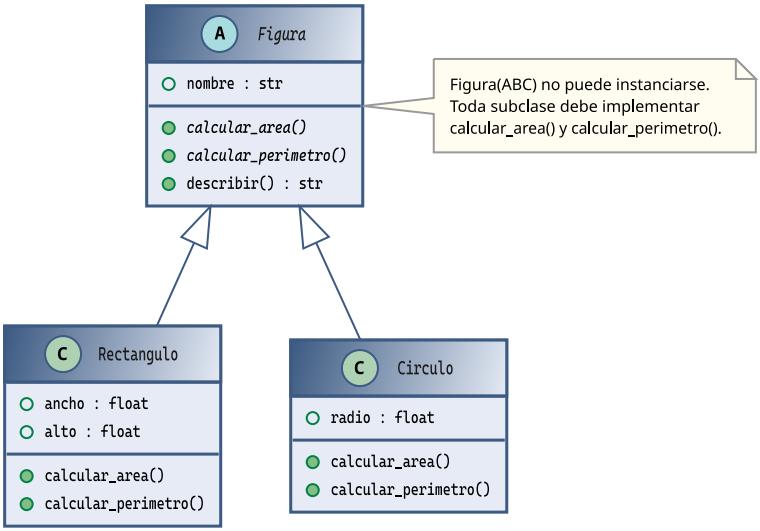
11.2.1 Concepto

Una clase abstracta es aquella que no puede ser instanciada directamente y que generalmente contiene uno o más métodos abstractos. Los métodos abstractos son declaraciones de métodos que deben ser implementados obligatoriamente por las clases derivadas, pero que no poseen implementación en la clase base.

Las clases abstractas sirven como plantillas o contratos que definen qué métodos deben implementar las clases derivadas, garantizando así una interfaz común entre diferentes implementaciones. Esto es especialmente útil cuando se desea definir un comportamiento común pero la implementación específica varía según cada subclase.

11.2.2 Uso del Decorador @abstractmethod

Python proporciona el módulo `abc` (Abstract Base Classes) para implementar clases y métodos abstractos. Para crear un método abstracto, se utiliza el decorador `@abstractmethod` y la clase debe heredar de `ABC`.




```python
from abc import ABC, abstractmethod

class Figura(ABC):
    def __init__(self, nombre):
        self.nombre = nombre

    @abstractmethod
    def calcular_area(self):
        pass

    @abstractmethod
    def calcular_perimetro(self):
        pass

    def describir(self):
        return f"Figura: {self.nombre}"

class Rectangulo(Figura):
    def __init__(self, ancho, alto):
        super().__init__("Rectángulo")
        self.ancho = ancho
        self.alto = alto

    def calcular_area(self):
        return self.ancho * self.alto

    def calcular_perimetro(self):
        return 2 * (self.ancho + self.alto)

class Circulo(Figura):
    def __init__(self, radio):
        super().__init__("Círculo")
        self.radio = radio
```

```
    def calcular_area(self):
        return 3.14159 * self.radio ** 2

    def calcular_perimetro(self):
        return 2 * 3.14159 * self.radio
```

Si se intenta instanciar la clase `Figura` directamente o crear una subclase que no implemente todos los métodos abstractos, Python generará un error `TypeError`. Esto garantiza que todas las clases derivadas cumplan con el contrato establecido por la clase abstracta.

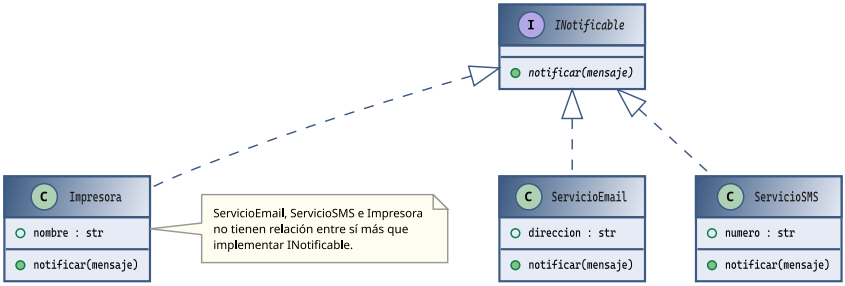
11.3 Interfaces

11.3.1 Implementación

Las interfaces definen un conjunto de métodos que una clase debe implementar, sin proporcionar la implementación de dichos métodos. En UML, las interfaces se representan como contratos que especifican qué servicios debe proporcionar una clase, pero no cómo debe proporcionarlos.

Aunque Python no tiene una palabra clave específica para interfaces como otros lenguajes, se pueden implementar utilizando clases abstractas que contengan únicamente métodos abstractos. Esta aproximación permite replicar el concepto de interfaces de UML en Python.

El verdadero valor de una interfaz no aparece cuando la implementa una sola clase, sino cuando la implementan varias clases que **no tienen ninguna otra relación entre sí** –no comparten una clase base común más allá de la interfaz, ni pertenecen al mismo dominio conceptual–. Lo único que las vincula es haberse comprometido con el mismo contrato, y eso alcanza para que un tercero pueda tratarlas de manera uniforme sin conocer su tipo concreto.




```python
from abc import ABC, abstractmethod

class INotificable(ABC):
    @abstractmethod
    def notificar(self, mensaje):
        pass

class ServicioEmail(INotificable):
    def __init__(self, direccion):
        self.direccion = direccion

    def notificar(self, mensaje):
        print(f"Enviando email a {self.direccion}: {mensaje}")

class ServicioSMS(INotificable):
    def __init__(self, numero):
        self.numero = numero

    def notificar(self, mensaje):
        print(f"Enviando SMS al {self.numero}: {mensaje}")

class Impresora(INotificable):
    def __init__(self, nombre):
        self.nombre = nombre

    def notificar(self, mensaje):
        print(f"Imprimiendo aviso en {self.nombre}: {mensaje}")
```

`ServicioEmail`, `ServicioSMS` e `Impresora` modelan tres cosas completamente distintas –un servicio de correo, una pasarela de SMS, un dispositivo de impresión– y no derivan unas de otras ni comparten ninguna clase base propia. Lo único que tienen en común es implementar `INotificable`. Gracias a eso, una función cliente puede recibir una colección heterogénea de objetos y notificarlos a todos por igual, sin necesidad de conocer de qué clase concreta es cada uno:

```python
def notificar_a_todos(destinatarios, mensaje):
    for destinatario in destinatarios:
        destinatario.notificar(mensaje)
```

```
destinatarios = [
    ServicioEmail("ana@empresa.com"),
    ServicioSMS("+54 9 351 555-0123"),
    Impresora("Impresora de Recepción"),
]

notificar_a_todos(destinatarios, "Reunión de personal a las 15hs")
```

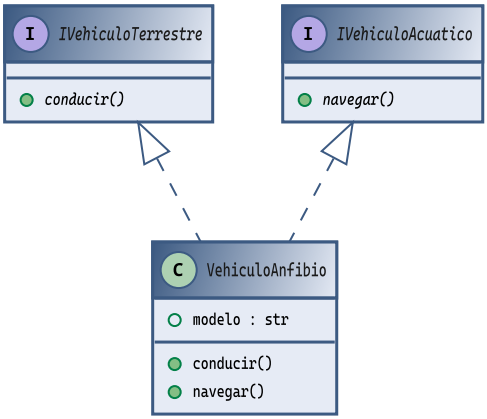
La función `notificar_a_todos` no necesita ningún `if` ni `isinstance` para distinguir entre los tres tipos de destinatario: le alcanza con saber que todos implementan `INotificable`, y por lo tanto todos responden al mensaje `notificar`. Esta es la idea central de las interfaces: permiten construir código que opera sobre el contrato, completamente desacoplado de las clases concretas que lo cumplen.

11.3.2 Implementación de múltiples interfaces

Python permite que una clase implemente más de una interfaz al mismo tiempo, indicando varias clases base entre paréntesis, tal como se hace en herencia. Esto podría parecer, a primera vista, un caso de herencia múltiple, pero conceptualmente es un mecanismo distinto y considerablemente más seguro.

La herencia múltiple propiamente dicha ocurre cuando dos o más clases base aportan **implementación** (atributos, cuerpos de métodos concretos) a una misma clase derivada. En ese escenario surge el llamado **problema del diamante**: si dos clases base definen un método con el mismo nombre pero distinta implementación, no existe una forma no ambigua de decidir cuál de las dos se hereda. Este es, precisamente, el motivo por el cual otros lenguajes no permiten que una clase extienda más de una clase concreta.

Sin embargo, cuando lo que se declara entre paréntesis no son clases con implementación sino **interfaces** –ABCs compuestas únicamente por métodos abstractos, como `IVehiculoTerrestre` e `IVehiculoAcuatico` en el ejemplo siguiente–, no hay ninguna implementación en conflicto que heredar: cada interfaz aporta solamente un contrato (qué métodos deben existir), nunca un cuerpo concreto que pueda entrar en colisión con el de otra.




```python
class IVehiculoTerrestre(ABC):
    @abstractmethod
    def conducir(self):
        pass

class IVehiculoAcuatico(ABC):
    @abstractmethod
    def navegar(self):
        pass

class VehiculoAnfibio(IVehiculoTerrestre, IVehiculoAcuatico):
    def __init__(self, modelo):
        self.modelo = modelo

    def conducir(self):
        print(f"{self.modelo} está conduciendo en tierra")

    def navegar(self):
        print(f"{self.modelo} está navegando en agua")
```

Esta implementación permite que un objeto `VehiculoAnfibio` sea tratado polimórficamente como un `IVehiculoTerrestre` o como un `IVehiculoAcuatico`, dependiendo del contexto en el que se utilice, replicando así el comportamiento de las interfaces múltiples de UML en Python.



Testing unitario

12.1 Introducción

El testing unitario es una práctica fundamental en el desarrollo de software que consiste en escribir y ejecutar pruebas automatizadas para verificar que las unidades individuales de código funcionen correctamente de manera aislada. Una unidad de código típicamente corresponde a un método, función o clase, y representa la porción más pequeña de código que puede ser probada de forma independiente.

El propósito principal del testing unitario es detectar errores en las primeras etapas del desarrollo, cuando son más fáciles y económicos de corregir. Al verificar que cada componente individual funciona según lo esperado, se puede tener mayor confianza en que el sistema completo funcionará correctamente cuando todas las partes se integren.

En la programación orientada a objetos, el testing unitario cobra especial relevancia debido a la naturaleza modular y encapsulada de las clases. Cada clase debe cumplir con un conjunto específico de responsabilidades y mantener su estado interno de manera coherente. Las pruebas unitarias permiten verificar que los métodos de una clase produzcan los resultados esperados, que las validaciones internas funcionen correctamente, y que el encapsulamiento se mantenga adecuadamente.

Los beneficios del testing unitario incluyen la detección temprana de errores, la facilitación del refactoring seguro del código, la documentación implícita del comportamiento esperado de cada componente, y la reducción

del tiempo necesario para localizar y corregir defectos. Además, las pruebas unitarias actúan como una red de seguridad que permite a los desarrolladores modificar el código con confianza, sabiendo que cualquier regresión será detectada inmediatamente.

Para que el testing unitario sea efectivo, las pruebas deben ser rápidas de ejecutar, independientes entre sí, repetibles y fáciles de mantener. Cada prueba debe verificar un comportamiento específico y proporcionar información clara sobre qué aspecto del código falló cuando ocurre un error.

12.2 Pytest

Pytest es un framework de testing para Python que se ha convertido en el estándar de facto para escribir y ejecutar pruebas unitarias en este lenguaje. A diferencia de otros frameworks de testing, pytest se destaca por su sintaxis simple y natural, su capacidad de autodescubrimiento de pruebas, y sus potentes características avanzadas que facilitan la escritura de pruebas comprehensivas.

Una de las principales ventajas de pytest es que permite escribir pruebas utilizando la sintaxis estándar de Python con la instrucción `assert`, eliminando la necesidad de aprender métodos específicos del framework como ocurre con otros sistemas de testing. Esto hace que las pruebas sean más legibles y fáciles de escribir para desarrolladores que ya conocen Python.

Para instalar pytest, se utiliza el gestor de paquetes pip mediante el siguiente comando:

```
pip install pytest
```

Una vez instalado, pytest puede ejecutarse desde la línea de comandos. El framework automáticamente descubre y ejecuta todas las pruebas en el directorio actual y sus subdirectorios. Para ejecutar las pruebas, simplemente se utiliza:

```
pytest
```

Pytest busca automáticamente archivos que comiencen con `test_` o terminen con `_test.py`, y dentro de estos archivos ejecuta todas las funciones que comiencen con `test_`. Esta convención de nomenclatura facilita la organización

y el descubrimiento automático de las pruebas.

Además del comando básico, pytest ofrece múltiples opciones de ejecución. Por ejemplo, se puede ejecutar un archivo específico de pruebas, mostrar información más detallada sobre la ejecución, o ejecutar solo las pruebas que contienen una palabra clave específica:

```
# Ejecutar un archivo específico
pytest test_ejemplo.py

# Mostrar información detallada
pytest -v

# Ejecutar pruebas que contengan una palabra clave
pytest -k "suma"
```

Pytest también genera reportes claros y detallados cuando las pruebas fallan, mostrando exactamente qué assertion falló, los valores esperados versus los obtenidos, y el contexto completo del error.

12.3 Pruebas

La estructura fundamental de las pruebas en pytest se basa en funciones que contienen assertions para verificar el comportamiento esperado del código. Una suite de pruebas es simplemente un conjunto de funciones de prueba organizadas en uno o más archivos, mientras que cada caso de prueba individual corresponde a una función específica que verifica un aspecto particular del código bajo prueba.

La sintaxis básica para escribir una prueba en pytest es crear una función cuyo nombre comience con `test_` y utilizar la instrucción `assert` para verificar condiciones:

```
def test_ejemplo_basico():
    resultado = 2 + 2
    assert resultado == 4
```

Para organizar las pruebas relacionadas con una clase específica, se recomienda crear un archivo de pruebas separado, típicamente con el nombre `test_nombre_clase.py`. Dentro de este archivo, se pueden agrupar las pruebas utilizando clases de prueba, aunque esto no es obligatorio:

```
class TestCalculadora:
    def test_suma(self):
        calc = Calculadora()
        resultado = calc.sumar(2, 3)
        assert resultado == 5

    def test_division_por_cero(self):
        calc = Calculadora()
        with pytest.raises(ZeroDivisionError):
            calc.dividir(10, 0)
```

12.3.1 Aserciones de igualdad

Las aserciones de igualdad son las más comunes en las pruebas unitarias y verifican que un valor obtenido sea exactamente igual al valor esperado. Pytest utiliza la instrucción `assert` estándar de Python junto con el operador de igualdad:

```
def test_igualdad_numerica():
    assert 5 == 5
    assert 2.5 == 2.5

def test_igualdad_cadenas():
    assert "hola" == "hola"
    assert "Python".lower() == "python"

def test_igualdad_listas():
    assert [1, 2, 3] == [1, 2, 3]
    assert sorted([3, 1, 2]) == [1, 2, 3]
```

Para la desigualdad, se utiliza el operador $\neq$:

```
def test_desigualdad():
    assert 5 ≠ 3
    assert "hola" ≠ "adiós"
```

12.3.2 Aserciones de comparación

Las aserciones de comparación permiten verificar relaciones de orden entre valores utilizando los operadores de comparación estándar de Python:

```
def test_comparaciones_numericas():
    assert 5 > 3
    assert 2 < 10
    assert 5 ≥ 5
    assert 3 ≤ 7

def test_comparaciones_cadenas():
```

```
    assert "abc" < "def"  # Orden alfabético
    assert "z" > "a"
```

12.3.3 Aserciones de pertenencia

Las aserciones de pertenencia verifican si un elemento está contenido en una colección utilizando el operador `in`:

```
def test_pertenencia_listas():
    numeros = [1, 2, 3, 4, 5]
    assert 3 in numeros
    assert 6 not in numeros

def test_pertenencia_cadenas():
    texto = "Hola mundo"
    assert "mundo" in texto
    assert "Python" not in texto

def test_pertenencia_diccionarios():
    datos = {"nombre": "Juan", "edad": 25}
    assert "nombre" in datos
    assert "apellido" not in datos
```

12.3.4 Aserciones de tipo

Las aserciones de tipo verifican que un objeto sea de un tipo específico utilizando la función `isinstance()`:

```
def test_tipos():
    assert isinstance(5, int)
    assert isinstance(3.14, float)
    assert isinstance("texto", str)
    assert isinstance([1, 2, 3], list)
    assert isinstance({"a": 1}, dict)
```

12.3.5 Aserciones de excepciones

Para verificar que el código lance excepciones bajo ciertas condiciones, pytest proporciona el context manager `pytest.raises()`:

```
import pytest

def test_excepcion_division_cero():
    with pytest.raises(ZeroDivisionError):
        resultado = 10 / 0

def test_excepcion_valor_incorrecto():
    with pytest.raises(ValueError):
```

```
        int("texto_invalido")

def test_excepcion_con_mensaje():
    with pytest.raises(ValueError, match="invalid literal"):
        int("abc")
```

12.3.6 Aserciones de aproximación

Para valores de punto flotante, donde la comparación exacta puede fallar debido a imprecisiones de representación, pytest ofrece `pytest.approx()`:

```
import pytest

def test_aproximacion():
    assert 0.1 + 0.2 == pytest.approx(0.3)
    assert 3.14159 == pytest.approx(3.14, rel=1e-2)
```

12.4 Ejemplo

Para ilustrar la aplicación práctica del testing unitario con pytest, se presenta una clase `Numeros` que encapsula una colección de números y proporciona métodos para realizar operaciones estadísticas básicas. Esta clase implementa el principio de composición al contener una lista de números como atributo interno.

```
class Numeros:
    def __init__(self):
        self._numeros = []

    def agregar(self, numero):
        if not isinstance(numero, (int, float)):
            raise TypeError("Solo se pueden agregar números")
        self._numeros.append(numero)

    def cantidad(self):
        return len(self._numeros)

    def suma(self):
        return sum(self._numeros)

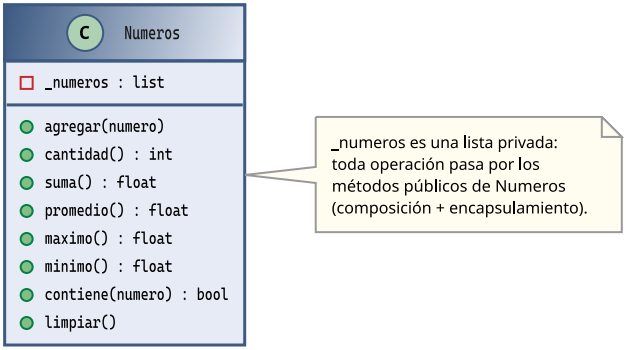
    def promedio(self):
        if self.cantidad() == 0:
            raise ValueError("No se puede calcular el promedio de una colección vacía")
        return self.suma() / self.cantidad()

    def maximo(self):
        if self.cantidad() == 0:
            raise ValueError("No se puede obtener el máximo de una colección vacía")
        return max(self._numeros)
```

```
    def minimo(self):
        if self.cantidad() == 0:
            raise ValueError("No se puede obtener el mínimo de una colección vacía")
        return min(self._numeros)

    def contiene(self, numero):
        return numero in self._numeros

    def limpiar(self):
         self._numeros.clear()
```



A continuación se presenta una suite completa de pruebas para esta clase, organizada de manera sistemática para cubrir todos los casos de uso posibles, incluyendo casos límite y situaciones de error:

```
import pytest
from numeros import Numeros

class TestNumeros:

    def test_coleccion_vacia_inicial(self):
        """Verifica que una nueva instancia esté vacía"""
        coleccion = Numeros()
        assert coleccion.cantidad() == 0

    def test_agregar_numero_entero(self):
        """Verifica que se puedan agregar números enteros"""
        coleccion = Numeros()
        coleccion.agregar(5)
        assert coleccion.cantidad() == 1
        assert coleccion.contiene(5)

    def test_agregar_numero_flotante(self):
        """Verifica que se puedan agregar números flotantes"""
        coleccion = Numeros()
        coleccion.agregar(3.14)
        assert coleccion.cantidad() == 1
        assert coleccion.contiene(3.14)
```

```python
    def test_agregar_numero_negativo(self):
        """Verifica que se puedan agregar números negativos"""
        coleccion = Numeros()
        coleccion.agregar(-10)
        assert coleccion.cantidad() == 1
        assert coleccion.contiene(-10)

    def test_agregar_cero(self):
        """Verifica que se pueda agregar el número cero"""
        coleccion = Numeros()
        coleccion.agregar(0)
        assert coleccion.cantidad() == 1
        assert coleccion.contiene(0)

    def test_agregar_tipo_invalido_cadena(self):
        """Verifica que se lance excepción al agregar una cadena"""
        coleccion = Numeros()
        with pytest.raises(TypeError, match="Solo se pueden agregar números"):
            coleccion.agregar("cinco")

    def test_agregar_tipo_invalido_lista(self):
        """Verifica que se lance excepción al agregar una lista"""
        coleccion = Numeros()
        with pytest.raises(TypeError):
            coleccion.agregar([1, 2, 3])

    def test_agregar_multiple_numeros(self):
        """Verifica que se puedan agregar múltiples números"""
        coleccion = Numeros()
        numeros = [1, 2, 3, 4, 5]
        for numero in numeros:
            coleccion.agregar(numero)

        assert coleccion.cantidad() == 5
        for numero in numeros:
            assert coleccion.contiene(numero)

    def test_suma_un_numero(self):
        """Verifica la suma con un solo número"""
        coleccion = Numeros()
        coleccion.agregar(7)
        assert coleccion.suma() == 7

    def test_suma_multiples_numeros(self):
        """Verifica la suma con múltiples números"""
        coleccion = Numeros()
        coleccion.agregar(2)
        coleccion.agregar(3)
        coleccion.agregar(5)
        assert coleccion.suma() == 10

    def test_suma_numeros_negativos(self):
        """Verifica la suma con números negativos"""
        coleccion = Numeros()
        coleccion.agregar(-2)
        coleccion.agregar(5)
        coleccion.agregar(-1)
        assert coleccion.suma() == 2

    def test_suma_coleccion_vacia(self):
```

```
        """Verifica que la suma de una colección vacía sea cero"""
        coleccion = Numeros()
        assert coleccion.suma() == 0

    def test_promedio_un_numero(self):
        """Verifica el promedio con un solo número"""
        coleccion = Numeros()
        coleccion.agregar(8)
        assert coleccion.promedio() == 8

    def test_promedio_multiples_numeros(self):
        """Verifica el promedio con múltiples números"""
        coleccion = Numeros()
        coleccion.agregar(2)
        coleccion.agregar(4)
        coleccion.agregar(6)
        assert coleccion.promedio() == 4

    def test_promedio_con_flotantes(self):
        """Verifica el promedio con números flotantes"""
        coleccion = Numeros()
        coleccion.agregar(1.5)
        coleccion.agregar(2.5)
        assert coleccion.promedio() == pytest.approx(2.0)

    def test_promedio_coleccion_vacia(self):
        """Verifica que se lance excepción al calcular promedio de colección vacía"""
        coleccion = Numeros()
        with pytest.raises(ValueError, match="No se puede calcular el promedio de una
        ↪  colección vacía"):
            coleccion.promedio()

    def test_maximo_un_numero(self):
        """Verifica el máximo con un solo número"""
        coleccion = Numeros()
        coleccion.agregar(42)
        assert coleccion.maximo() == 42

    def test_maximo_multiples_numeros(self):
        """Verifica el máximo con múltiples números"""
        coleccion = Numeros()
        numeros = [3, 1, 4, 1, 5, 9, 2, 6]
        for numero in numeros:
            coleccion.agregar(numero)
        assert coleccion.maximo() == 9

    def test_maximo_numeros_negativos(self):
        """Verifica el máximo con solo números negativos"""
        coleccion = Numeros()
        coleccion.agregar(-5)
        coleccion.agregar(-2)
        coleccion.agregar(-8)
        assert coleccion.maximo() == -2

    def test_maximo_coleccion_vacia(self):
        """Verifica que se lance excepción al buscar máximo de colección vacía"""
        coleccion = Numeros()
        with pytest.raises(ValueError, match="No se puede obtener el máximo de una
        ↪  colección vacía"):
            coleccion.maximo()
```

```python
    def test_minimo_un_numero(self):
        """Verifica el mínimo con un solo número"""
        coleccion = Numeros()
        coleccion.agregar(17)
        assert coleccion.minimo() == 17

    def test_minimo_multiples_numeros(self):
        """Verifica el mínimo con múltiples números"""
        coleccion = Numeros()
        numeros = [3, 1, 4, 1, 5, 9, 2, 6]
        for numero in numeros:
            coleccion.agregar(numero)
        assert coleccion.minimo() == 1

    def test_minimo_numeros_positivos(self):
        """Verifica el mínimo con solo números positivos"""
        coleccion = Numeros()
        coleccion.agregar(5)
        coleccion.agregar(2)
        coleccion.agregar(8)
        assert coleccion.minimo() == 2

    def test_minimo_coleccion_vacia(self):
        """Verifica que se lance excepción al buscar mínimo de colección vacía"""
        coleccion = Numeros()
        with pytest.raises(ValueError, match="No se puede obtener el mínimo de una
        ↪  colección vacía"):
            coleccion.minimo()

    def test_contiene_numero_presente(self):
        """Verifica que contiene devuelva True para números presentes"""
        coleccion = Numeros()
        coleccion.agregar(10)
        coleccion.agregar(20)
        coleccion.agregar(30)
        assert coleccion.contiene(20)

    def test_contiene_numero_ausente(self):
        """Verifica que contiene devuelva False para números ausentes"""
        coleccion = Numeros()
        coleccion.agregar(10)
        coleccion.agregar(20)
        assert not coleccion.contiene(15)

    def test_contiene_coleccion_vacia(self):
        """Verifica que contiene devuelva False en colección vacía"""
        coleccion = Numeros()
        assert not coleccion.contiene(5)

    def test_limpiar_coleccion(self):
        """Verifica que limpiar vacíe completamente la colección"""
        coleccion = Numeros()
        coleccion.agregar(1)
        coleccion.agregar(2)
        coleccion.agregar(3)

        assert coleccion.cantidad() == 3

        coleccion.limpiar()
        assert coleccion.cantidad() == 0
```

```
        assert not coleccion.contiene(1)
        assert not coleccion.contiene(2)
        assert not coleccion.contiene(3)

    def test_operaciones_despues_limpiar(self):
        """Verifica que las operaciones fallen después de limpiar"""
        coleccion = Numeros()
        coleccion.agregar(5)
        coleccion.limpiar()

        with pytest.raises(ValueError):
            coleccion.promedio()

        with pytest.raises(ValueError):
            coleccion.maximo()

        with pytest.raises(ValueError):
            coleccion.minimo()

    def test_flujo_completo(self):
        """Prueba de integración que verifica un flujo completo de operaciones"""
        coleccion = Numeros()

        # Agregar números
        numeros = [2, 4, 6, 8, 10]
        for numero in numeros:
            coleccion.agregar(numero)

        # Verificar todas las operaciones
        assert coleccion.cantidad() == 5
        assert coleccion.suma() == 30
        assert coleccion.promedio() == 6
        assert coleccion.maximo() == 10
        assert coleccion.minimo() == 2

        # Verificar pertenencia
        for numero in numeros:
            assert coleccion.contiene(numero)

        assert not coleccion.contiene(7)
```

Esta suite de pruebas cubre exhaustivamente todos los métodos de la clase `Numeros`, incluyendo casos normales, casos límite y situaciones de error. Las pruebas están organizadas de manera lógica, cada una verifica un comportamiento específico, y utilizan nombres descriptivos que facilitan la comprensión de qué está siendo probado.

El conjunto de pruebas incluye verificaciones para el manejo correcto de excepciones, validaciones de tipos, operaciones con colecciones vacías, y un caso de prueba de integración que verifica el flujo completo de operaciones. Esta aproximación sistemática al testing garantiza que la clase `Numeros` funcione correctamente en todas las situaciones previstas.